

Introduction to Information Retrieval

Local Features

Lecturer: Dr. LE Trung Nghia, Dr. NGUYEN Vinh Tiep

Disclaimer

- Disclaimer: This curriculum have been inspired and adapted from materials created by:
 - Richard Szeliski, “Computer Vision: Algorithms and Applications”, 2nd edition, 2022. <https://szeliski.org/Book/>

Remind: Image Gradient

- Directional change in intensity or color in image
- Derivative of a multivariate function:

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

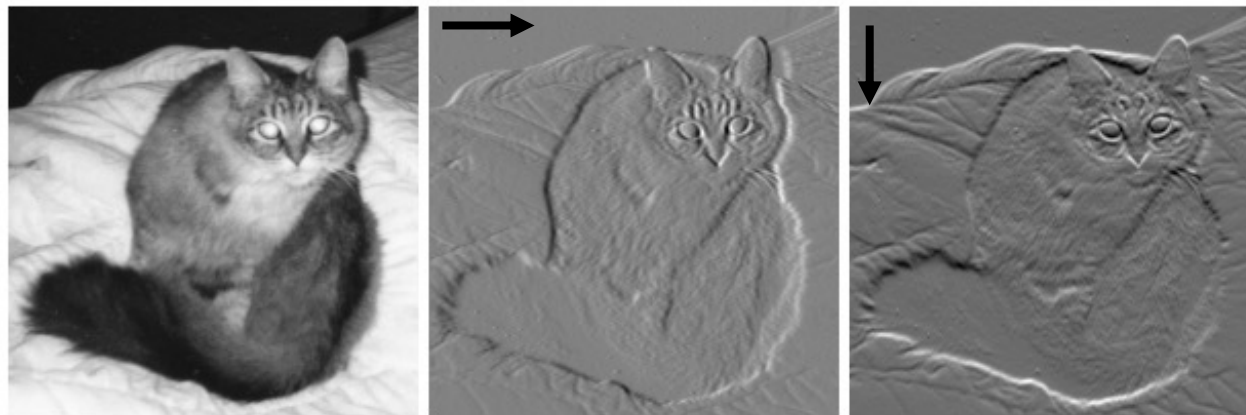
- Direction: $\theta = \tan^{-1} \begin{bmatrix} g_y \\ g_x \end{bmatrix}$
- Magnitude: $m = \sqrt{g_y^2 + g_x^2}$



Remind: Image Gradient

- For a discrete image, can be approximated with finite differences

$$\begin{aligned}\frac{df}{dx} &\approx f(x+1, y) - f(x, y) \\ \frac{df}{dy} &\approx f(x, y+1) - f(x, y)\end{aligned} \quad \Rightarrow \quad \begin{aligned}\frac{\partial f}{\partial x} &\approx [-1 \quad 1] * A = \text{conv}(A, [-1 \quad 1]) \\ \frac{\partial f}{\partial y} &\approx \begin{bmatrix} -1 \\ 1 \end{bmatrix} * A = \text{conv}\left(A, \begin{bmatrix} -1 \\ 1 \end{bmatrix}\right)\end{aligned}$$



Horizontal change

Vertical change

Feature Representation

- Global feature representations are built on taking a whole image as an input
- Local feature representations are made for each local region within an image

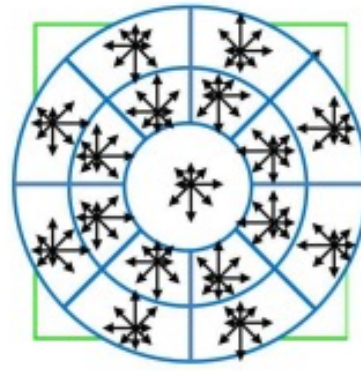


Why Local Features?

- Analyze, describe, and match images



(a)



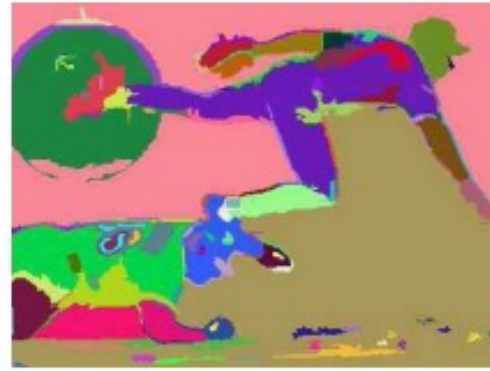
(b)



(c)



(d)



(e)



(f)

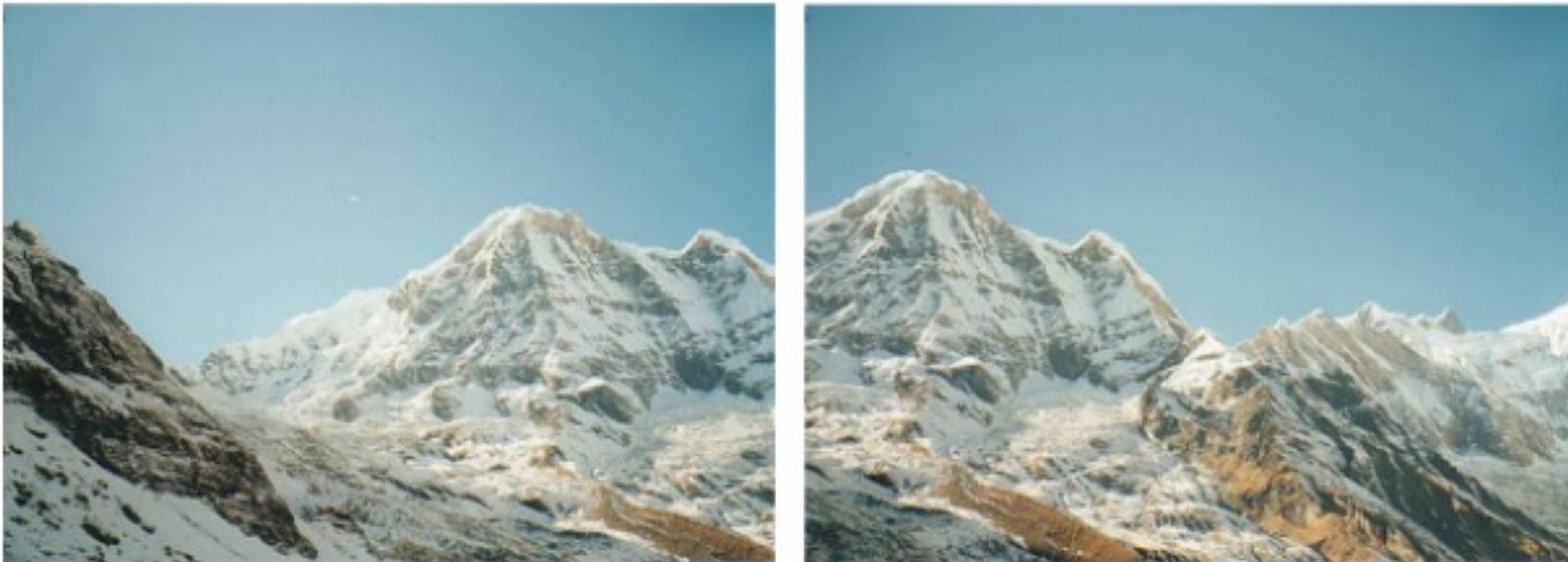
Why Local Features?

- Motivation: Image matching
 - We have two images taken by two tourists
 - How do we ensure that these two pictures are from the same tourist spot?



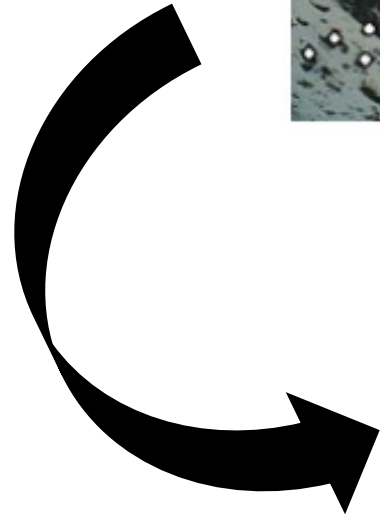
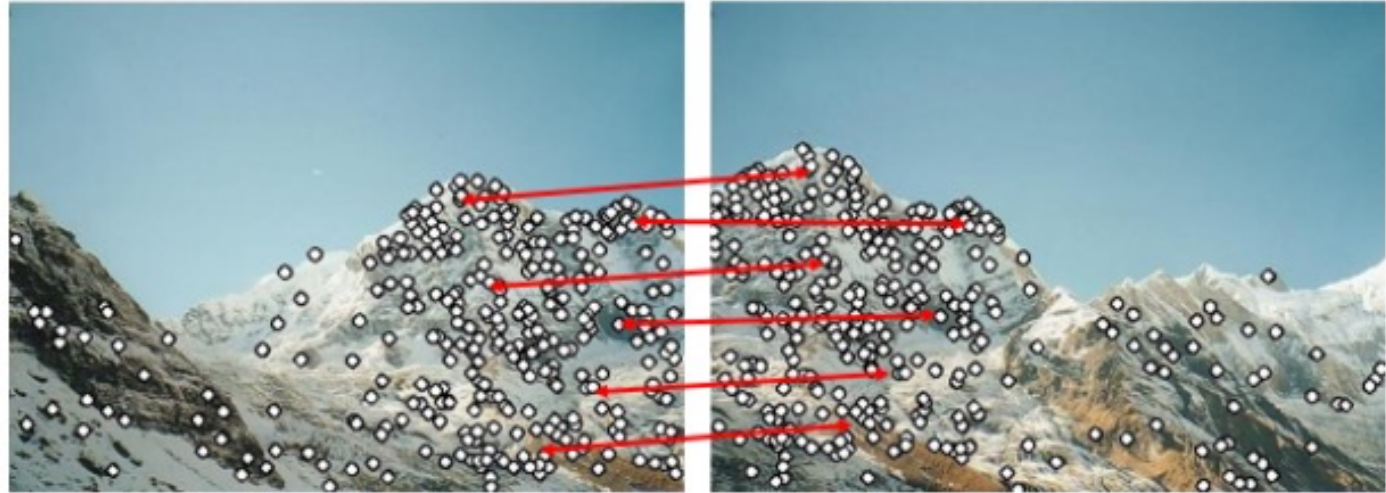
Why Local Features?

- Motivation: Panorama stitching
 - We have two image
 - How do we combine them?



Why Local Features?

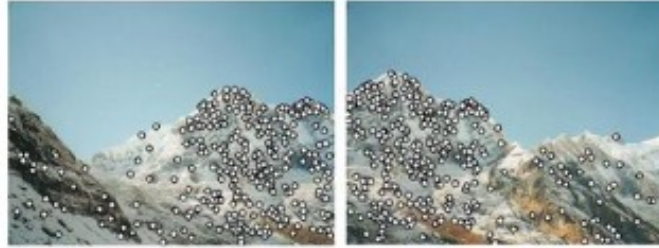
- Step 1: extract features
- Step 2: match features
- Step 3: align images



Main Components of Local Features

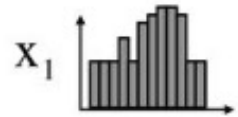
1) Detection:

Find a set of distinctive key points.

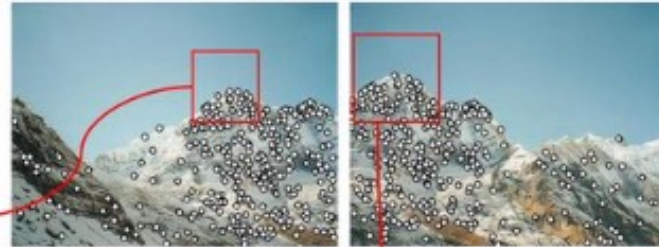


2) Description:

Extract feature descriptor around each interest point as vector.



$$\mathbf{x}_1 = [x_1^{(1)}, \dots, x_d^{(1)}]$$



$$\mathbf{x}_2 = [x_1^{(2)}, \dots, x_d^{(2)}]$$

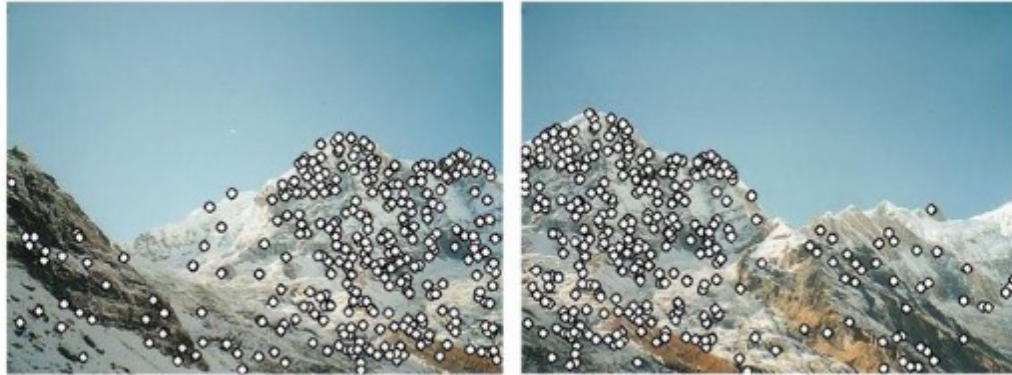
3) Matching:

Compute distance between feature vectors to find correspondence.

$$d(\mathbf{x}_1, \mathbf{x}_2) < T$$



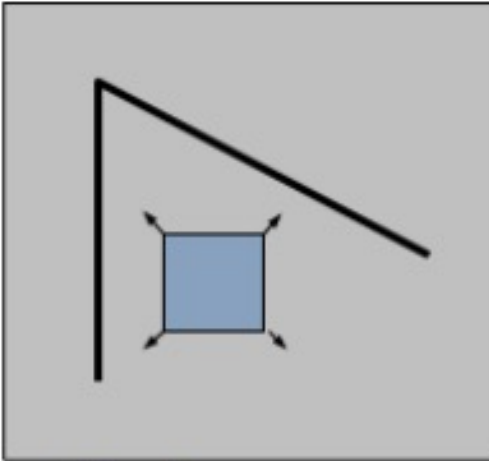
Characteristics of Good Features



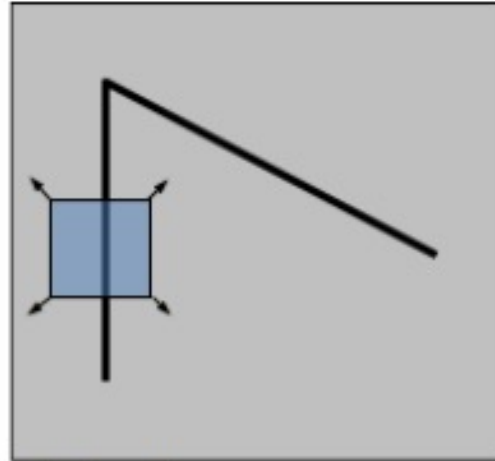
- **Repeatability**
 - The same feature can be found in several images despite geometric and photometric transformations
- **Saliency**
 - Each feature is distinctive
- **Compactness and efficiency**
 - Many fewer features than image pixels
- **Locality**
 - A feature occupies a relatively small area of the image; robust to clutter and occlusion

Local Measure of Feature Uniqueness

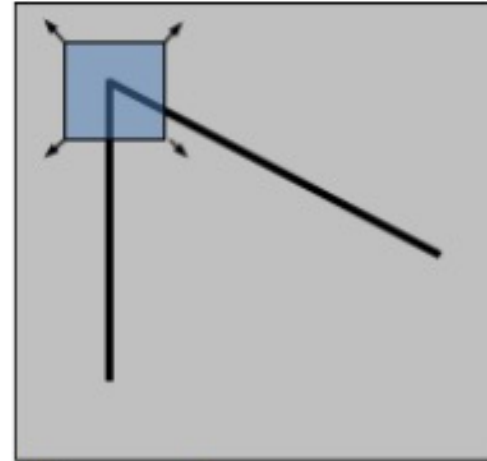
- How does the window change when you shift it?
- Shifting the window in any direction causes a big change



“flat” region:
no change in all
directions

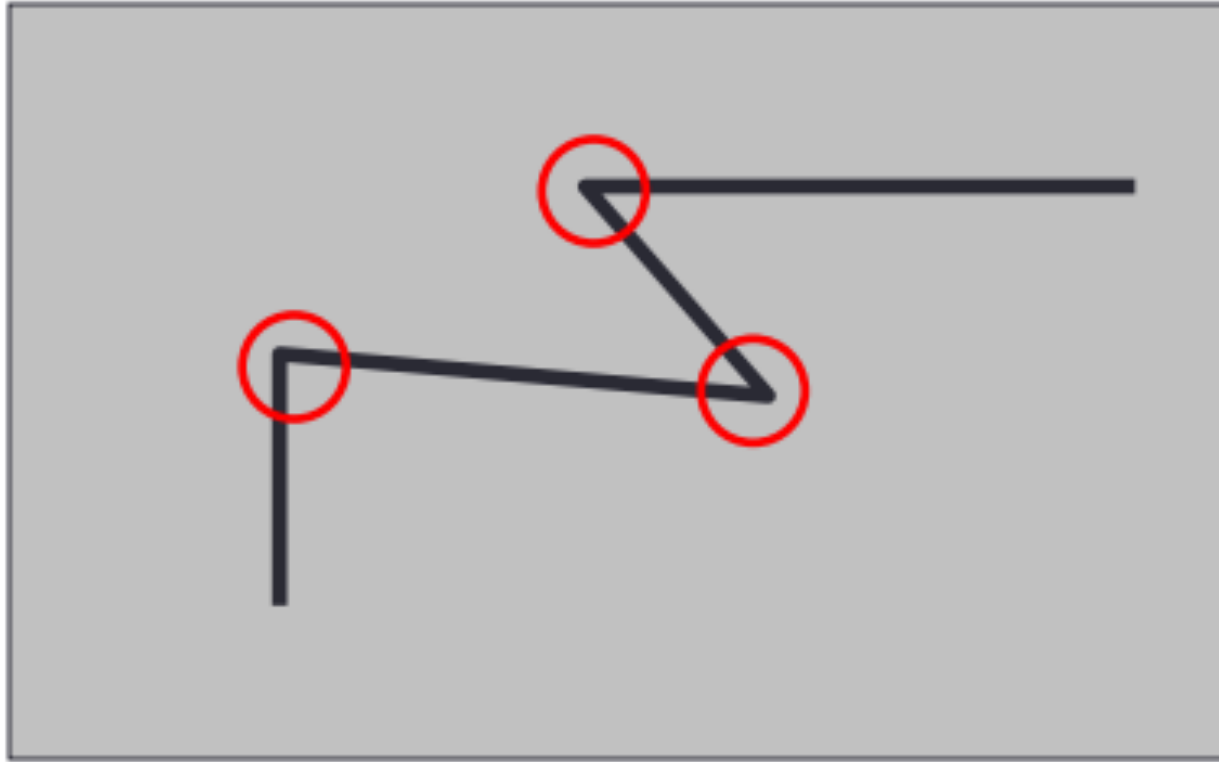


“edge”:
no change along the
edge direction



“corner”:
significant change in
all directions

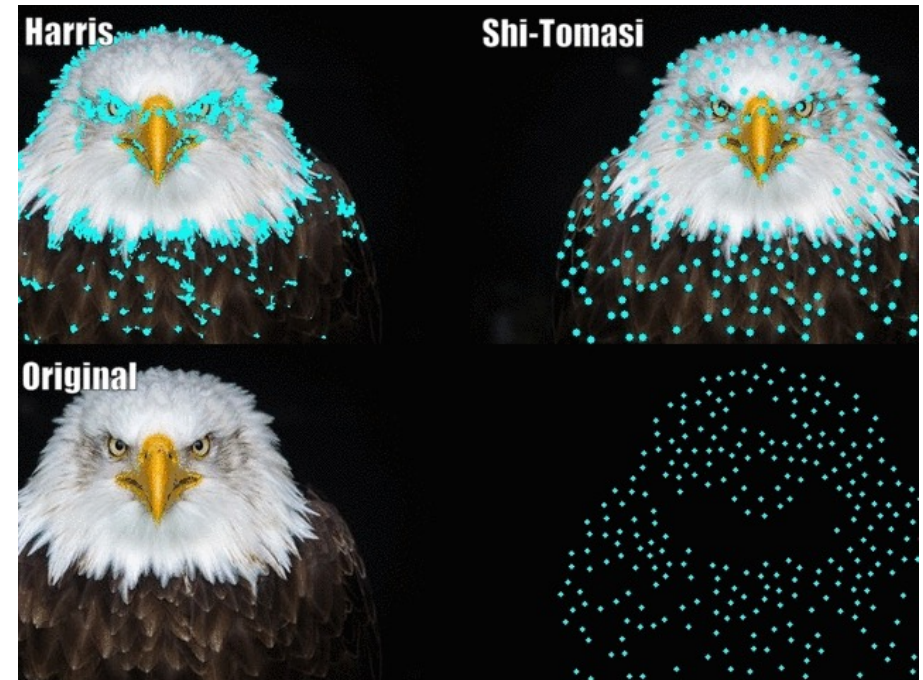
Harris Corner Detection (Harris et al., 1988)



Harris Corner Detection (Harris et al., 1988)

- Corners are regions in a given image with a large variation in intensity between pixels in all directions
- Find pixel-intensity displacement of (u, v) such that the function E gets maximized for pixels in the window

$$E(u, v) = \sum_{(x,y) \in W} [I(x+u, y+v) - I(x, y)]^2$$



Harris Corner Response Function

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Measure of corner response:

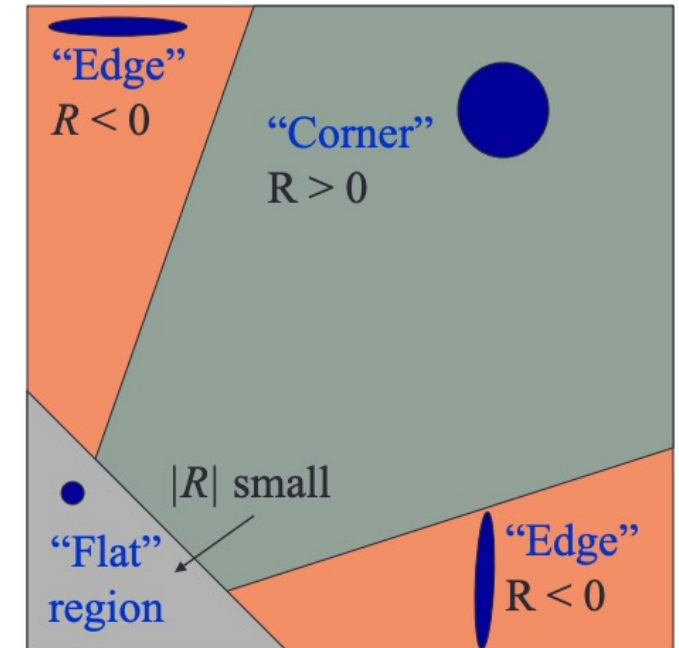
$$R = \det M - \alpha (\text{trace } M)^2$$

$$\begin{aligned} \det M &= \lambda_1 \lambda_2 \\ \text{trace } M &= \lambda_1 + \lambda_2 \end{aligned}$$

(α – empirical constant, typically 0.04 - 0.06)

$$R = \det(M) - \alpha \text{trace}(M)^2 = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

- R depends only on eigenvalues of M , but don't compute them (no sqrt, so really fast!)
- R is large for a **corner**
- R is negative with large magnitude for an **edge**
- $|R|$ is small for a **flat** region



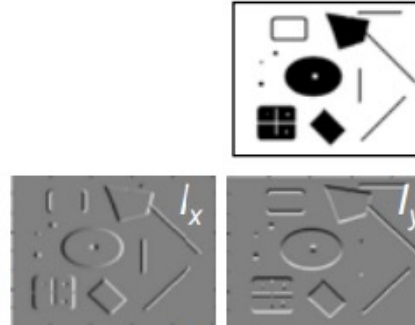
Steps of Harris Corner Detection

- Second moment

matrix

$$\mu(\sigma_I, \sigma_D) = g(\sigma_I) * \begin{bmatrix} I_x^2(\sigma_D) & I_x I_y(\sigma_D) \\ I_x I_y(\sigma_D) & I_y^2(\sigma_D) \end{bmatrix}$$

1. Image derivatives
(optionally, blur first)



`Cv2.Sobel(img, 1, 0)`

`Cv2.Sobel(img, 0, 1)`

`dx2=np.square(dx)`

`dy2=np.square(dy)`

`dxy=dx*dy`

2. Square of derivatives



3. Gaussian filter $g(\sigma_f)$



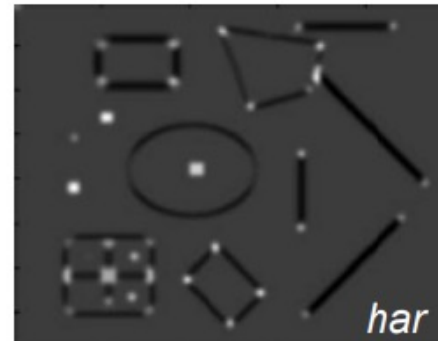
$$\det M = \lambda_1 \lambda_2$$

$$\text{trace } M = \lambda_1 + \lambda_2$$

4. Cornerness function – both eigenvalues are strong

$$har = \det[\mu(\sigma_I, \sigma_D)] - \alpha [\text{trace}(\mu(\sigma_I, \sigma_D))]^2 = g(I_x^2)g(I_y^2) - [g(I_x I_y)]^2 - \alpha [g(I_x^2) + g(I_y^2)]^2$$

5. Non-maxima suppression



$$r(harris) = \det - k * (\text{trace} ** 2)$$

`loc = np.where(harris >= threshold)`

Example

Raw Image



Sobel on X axis



Sobel on Y axis



Gauss on X axis



Gauss on Y axis



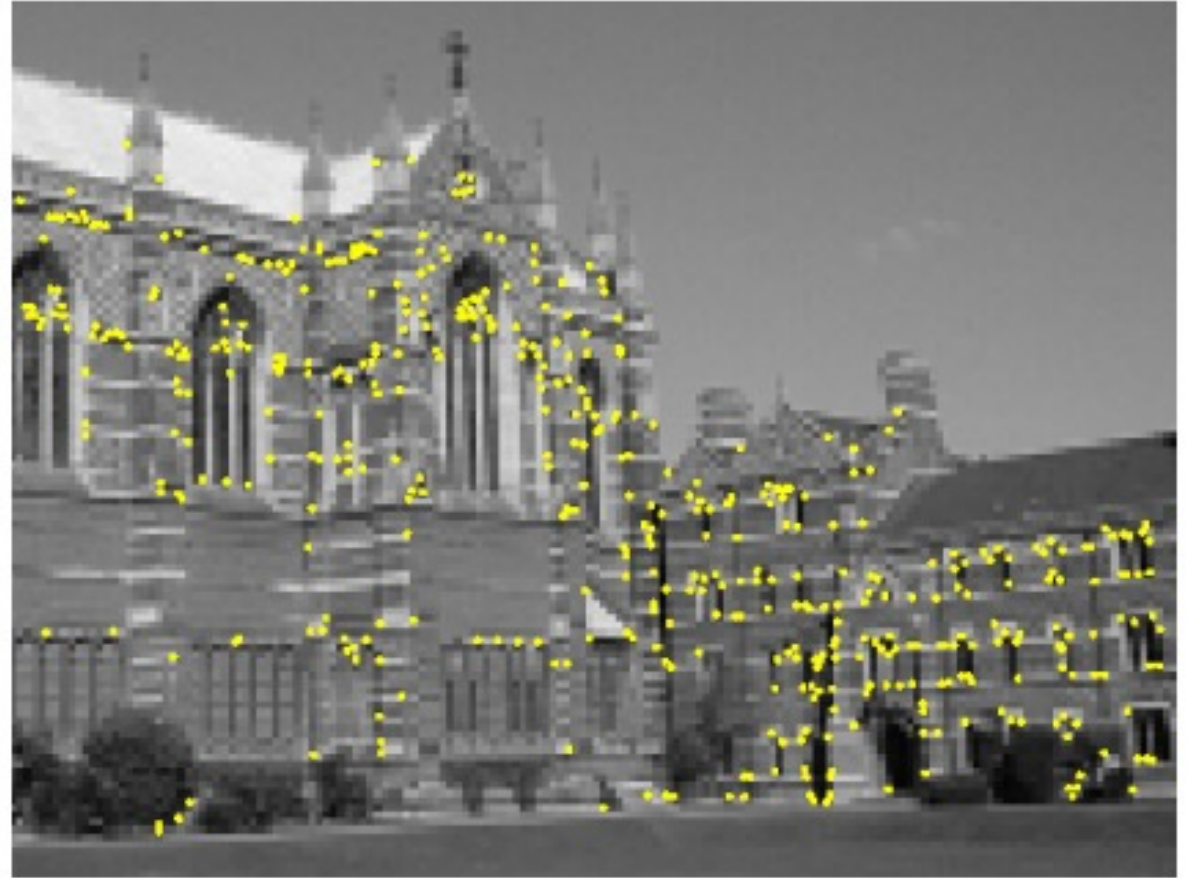
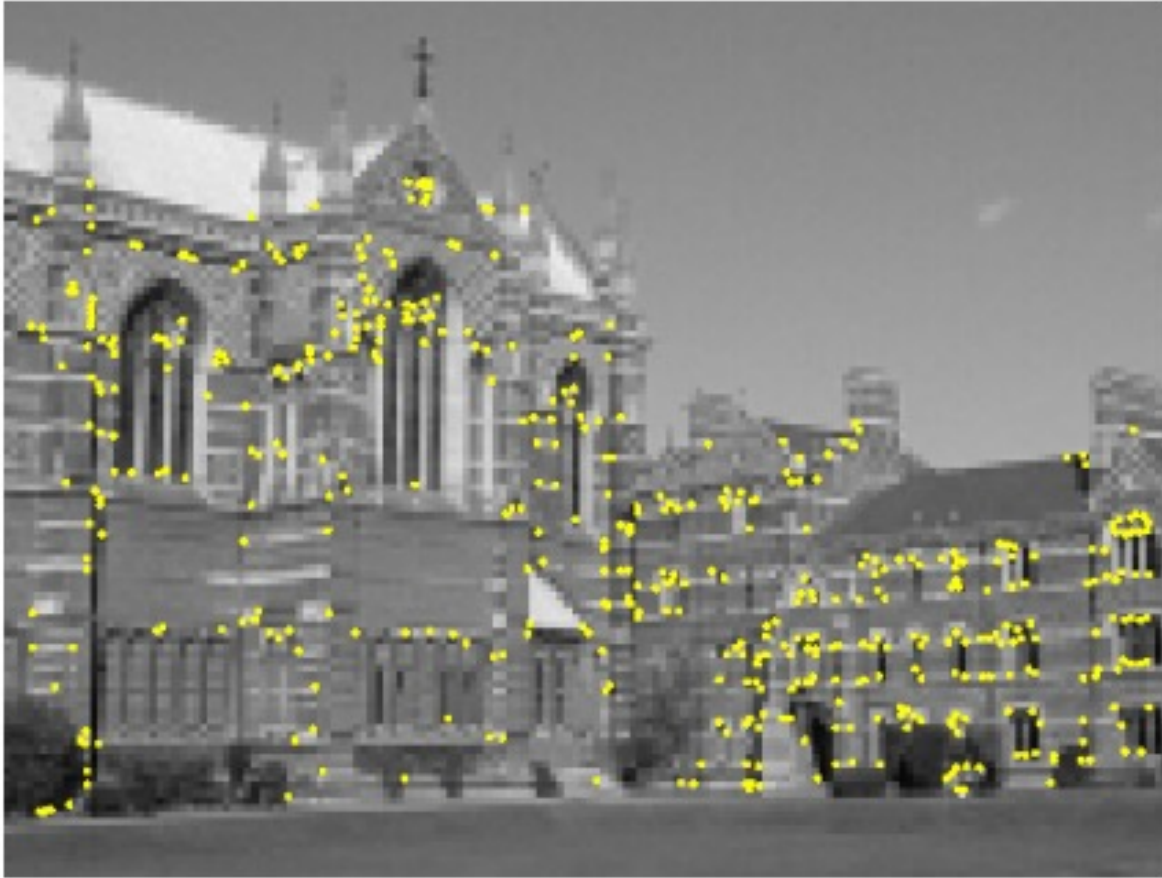
Raw Image



Harris Corner Output

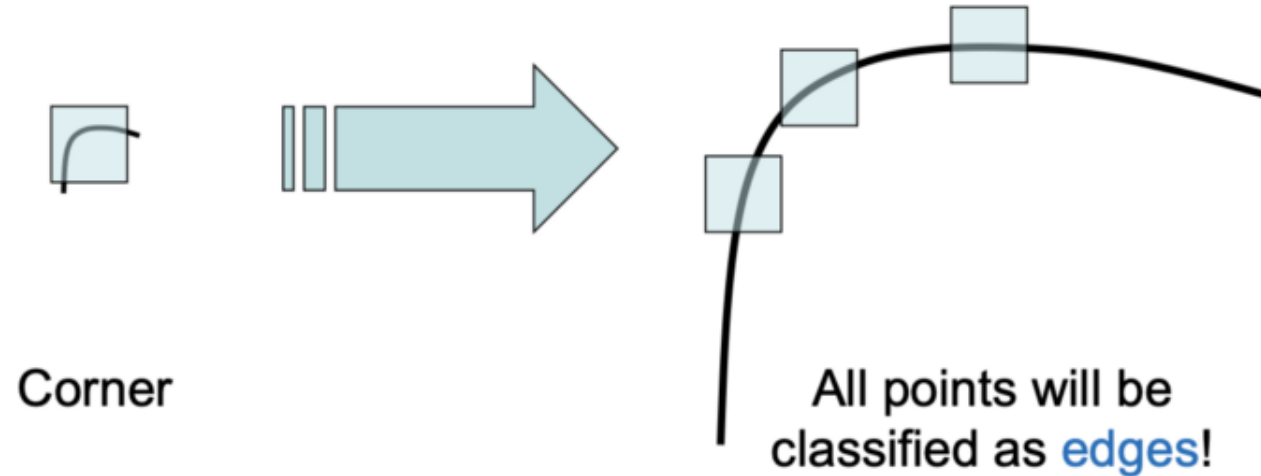


Example



Harris Corner Properties

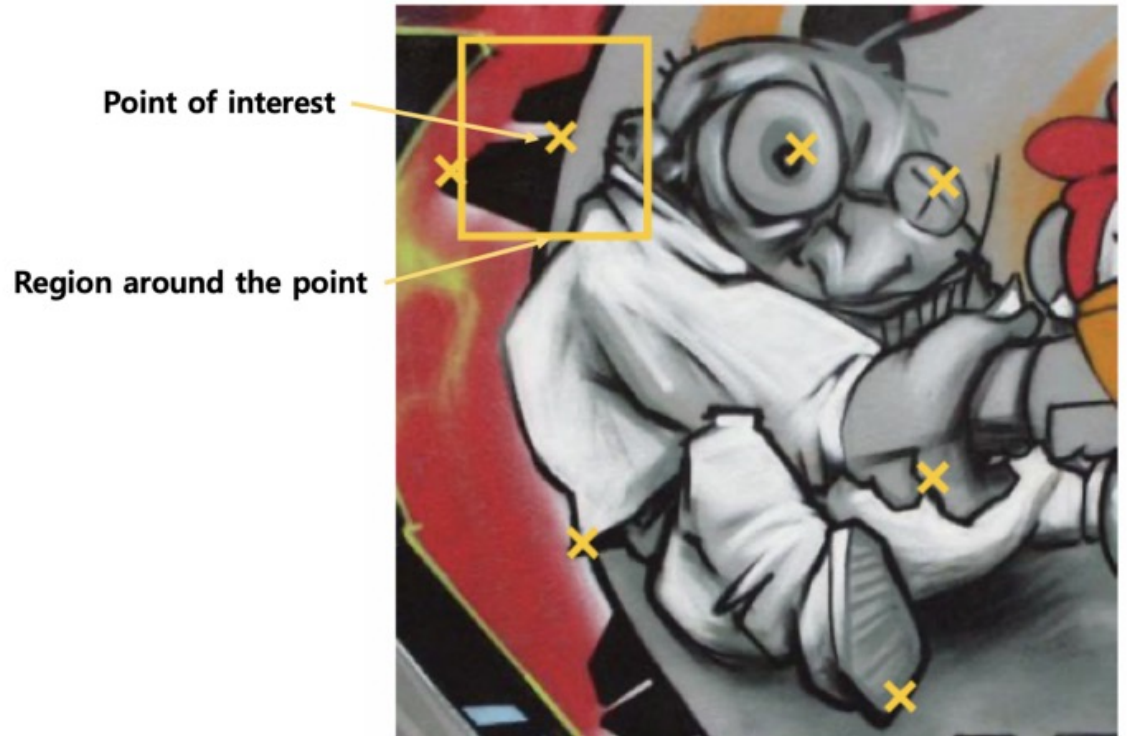
- Invariant to image rotation
- Partially invariant to affine intensity change
- Not invariant to scaling



Not invariant to image scale!

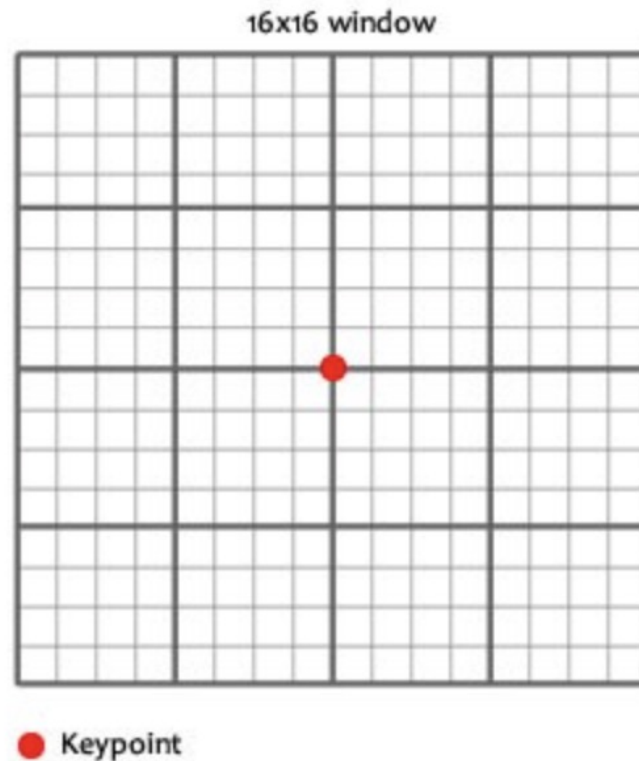
From Point to Region

- The Harris and Hessian operators return us the interest points (x, y) where corners are located. However, in order to define the feature descriptions (feature vector representations) for the interest points, and to compare the points between images of interest, but a region around it.



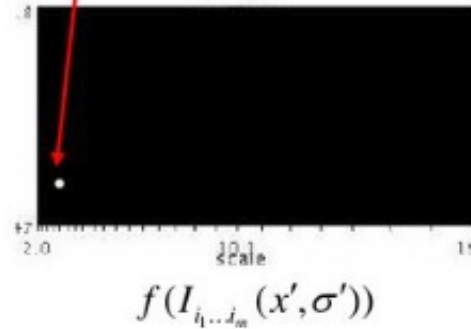
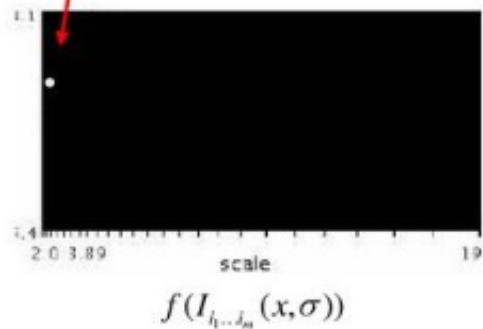
How to Extract Blob?

- The simplest way to define a region is to take 16×16 pixels around the detected point



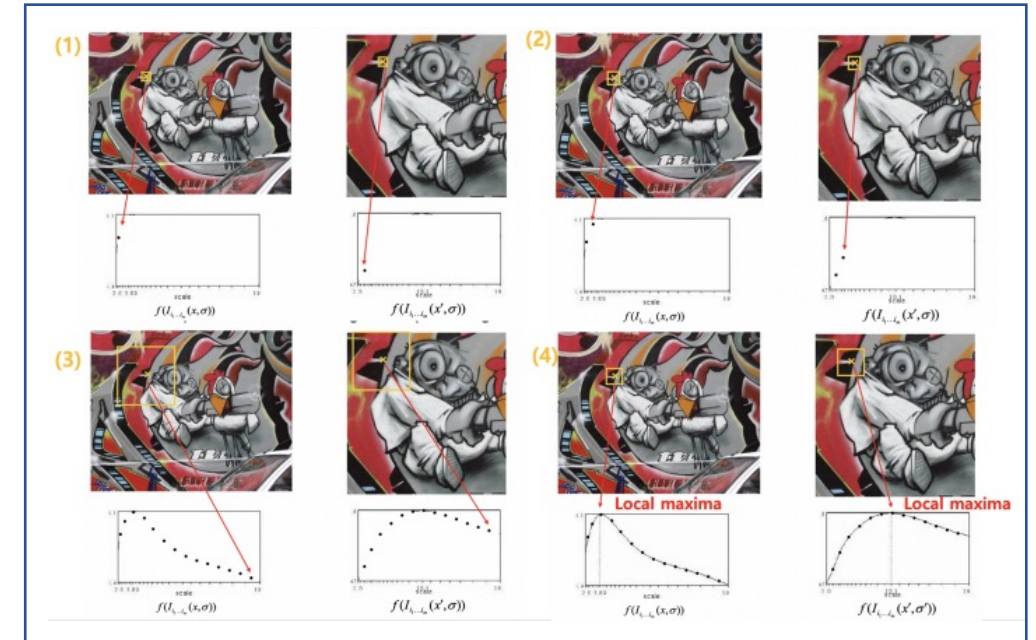
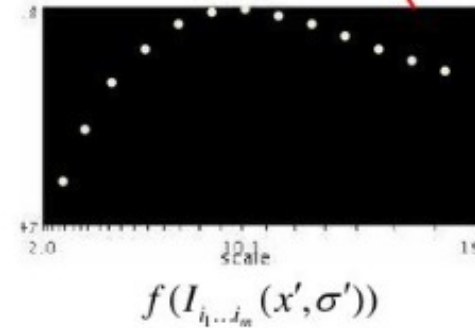
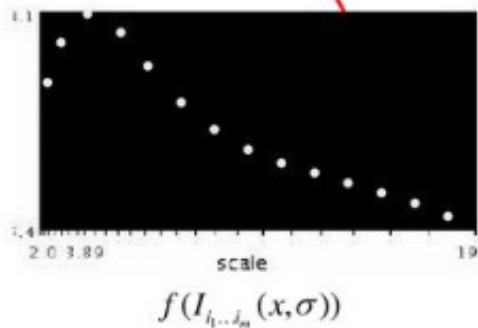
Automatic Scale Selection (Lindeberg et al., 1996)

- Function response for increasing scale (scale signature)



Automatic Scale Selection

- Function response for increasing scale (scale signature)

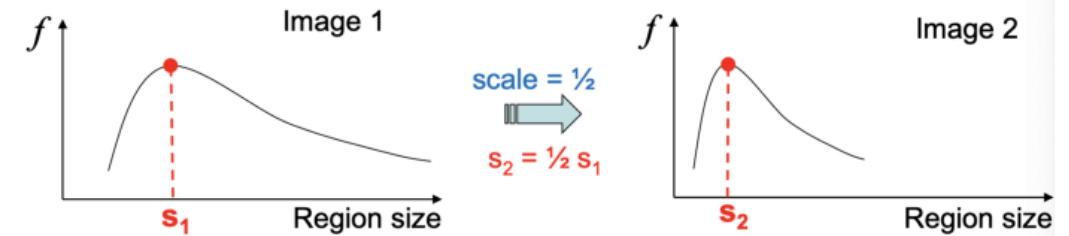


Automatic Scale Selection

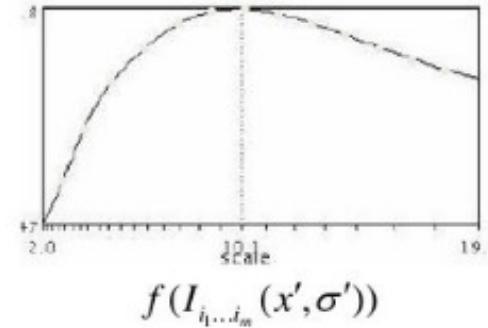
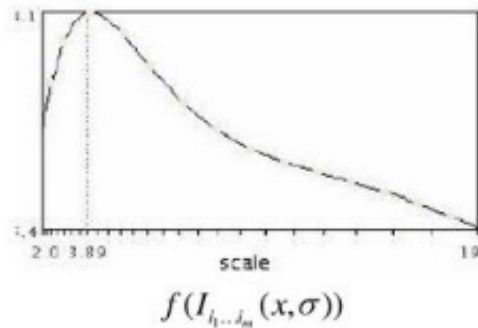
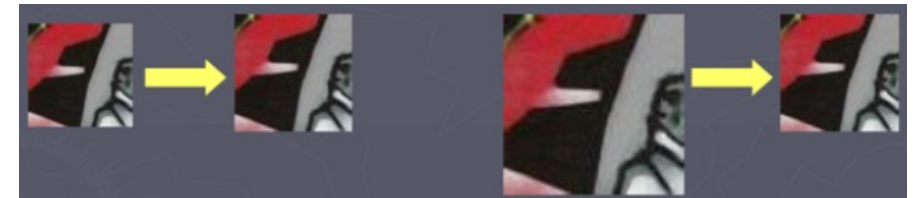
- Function response for increasing scale (scale signature)



Important: this scale invariant region size is found in each image **independently!**



Remember to normalize (rescale to fixed size)



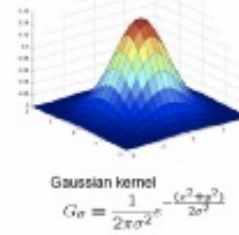
Laplacian-of-Gaussian (LoG)

- Blob detector

$$LoG = -\frac{1}{\pi\sigma^4} \left[1 - \frac{x^2 + y^2}{2\sigma^2} \right] e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

0	-1	0
-1	4	-1
0	-1	0

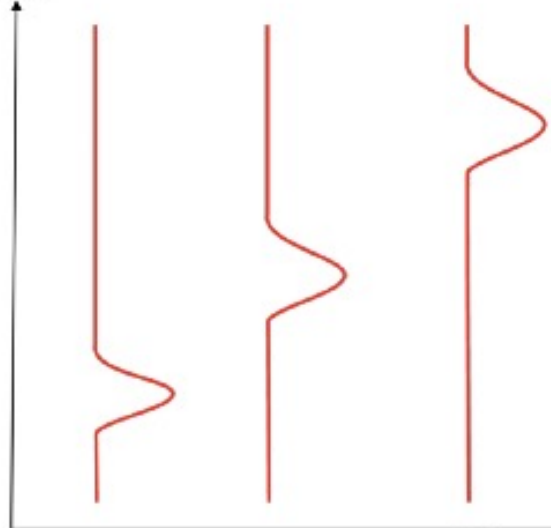
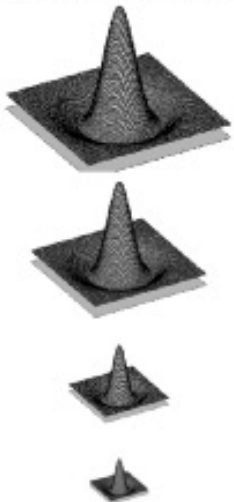
★



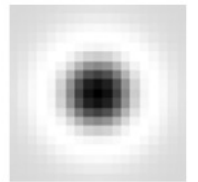
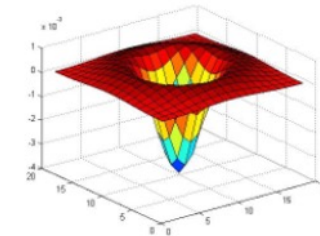
Laplacian Filter

Gaussian Filter

LoG for varying scales



Given pattern (circle) with varying sizes
(Image blob size)



$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$$

(very similar to a Difference of Gaussians (DoG) –
i.e. a Gaussian minus a slightly smaller Gaussian)

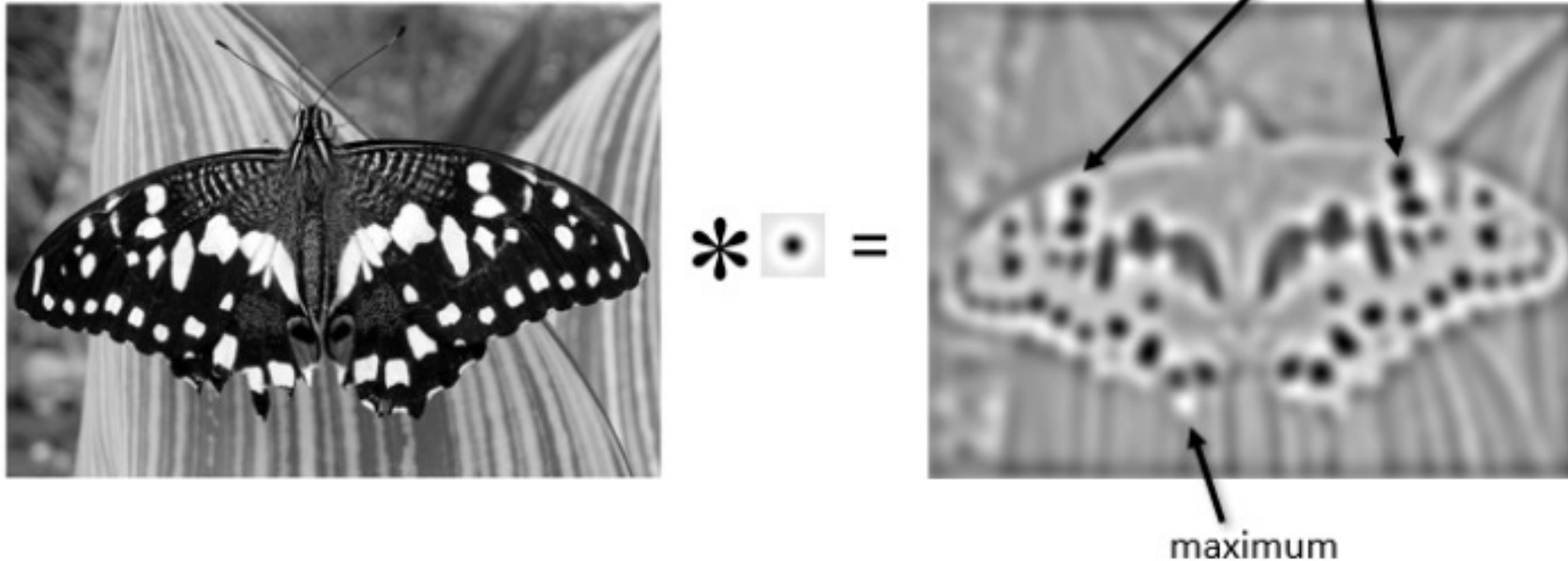
Difference-of-Gaussian (DoG)

- Approximate LoG with DoG:
 1. Blur image with σ Gaussian kernel
 2. Blur image with $k\sigma$ Gaussian kernel
 3. Subtract (2) from (1)

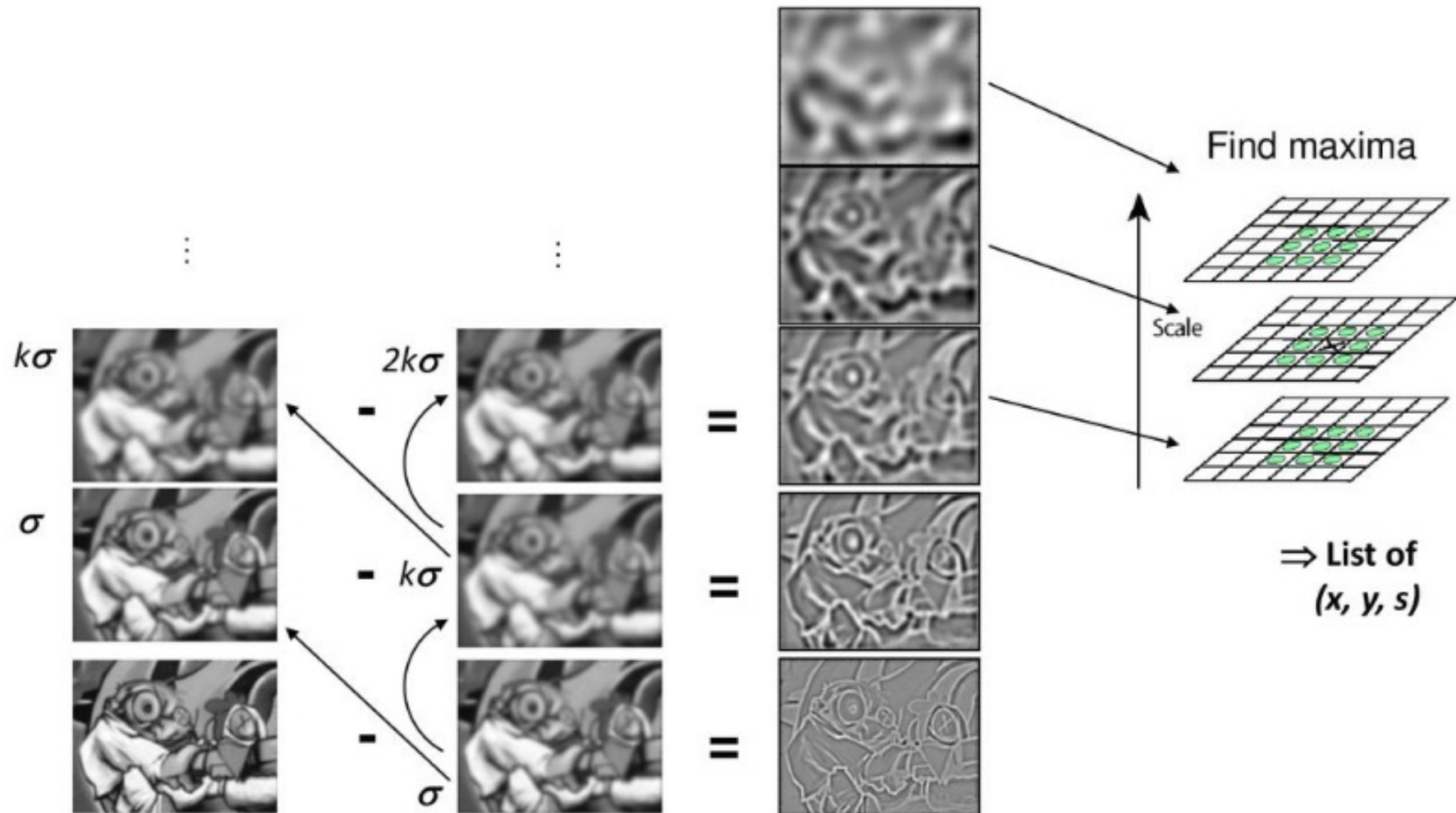


Difference-of-Gaussian (DoG)

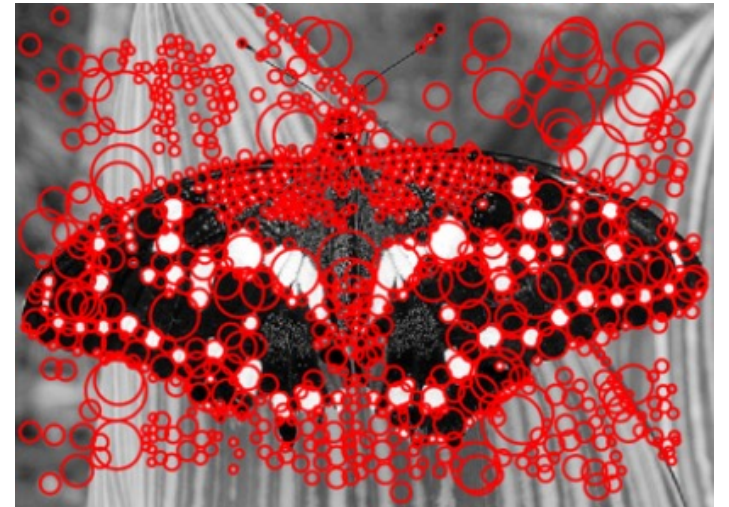
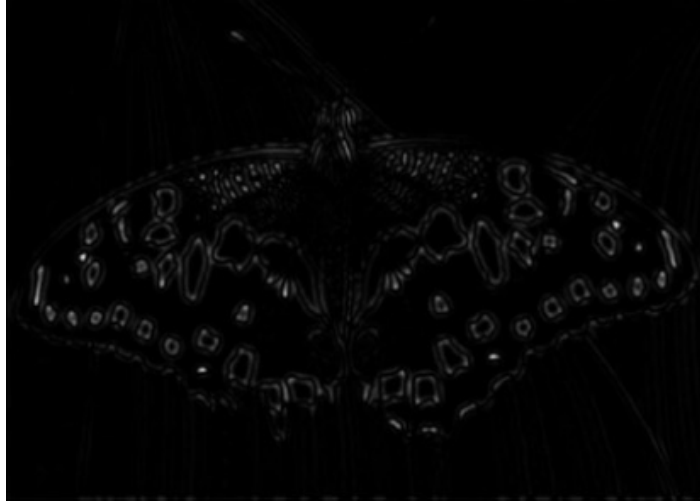
- Blob detector
- Find maxima and minima of LoG operator in space and scale
 - Maxima = dark blobs on light background
 - Minima = light blobs on dark background



Find Local Maxima in Position-Scale Space of DoG



Results of DoG



Invariant Local Features

- Find features that are invariant to transform:
 - Geometric transform: translation, rotation, scale
 - Photometric transform: brightness, exposure, ...

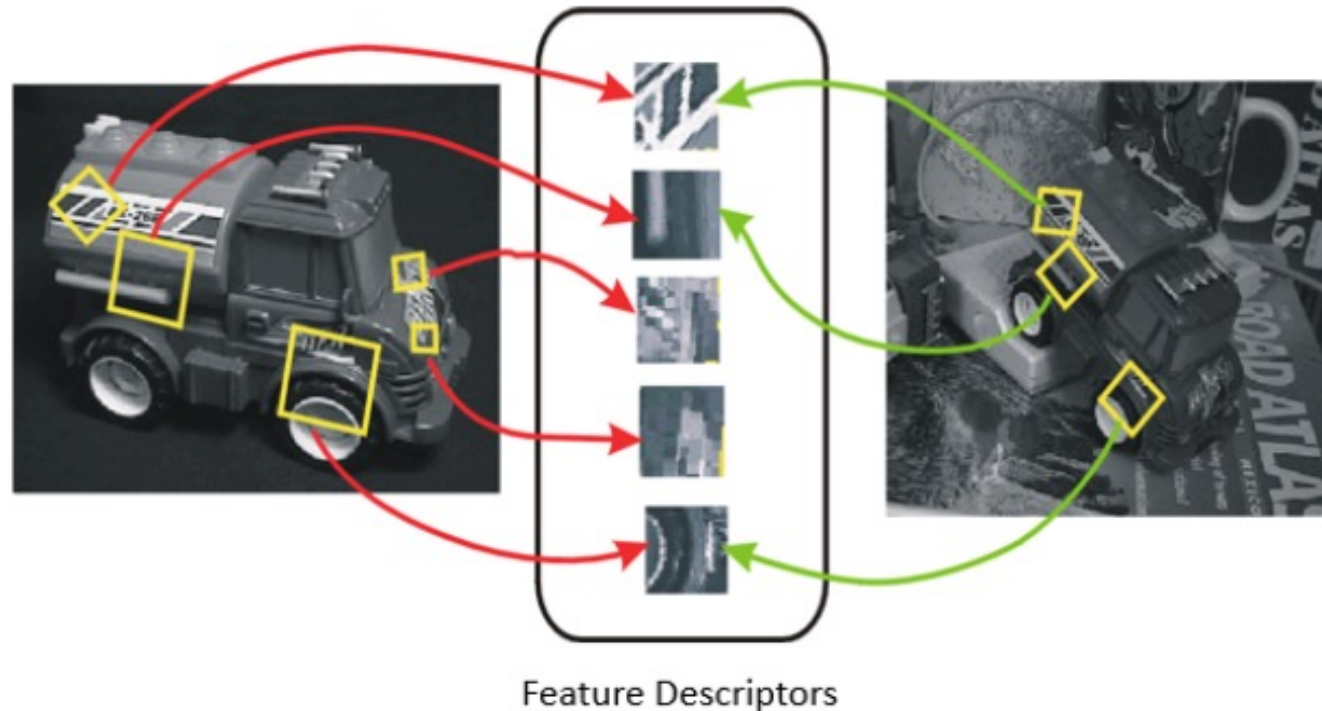
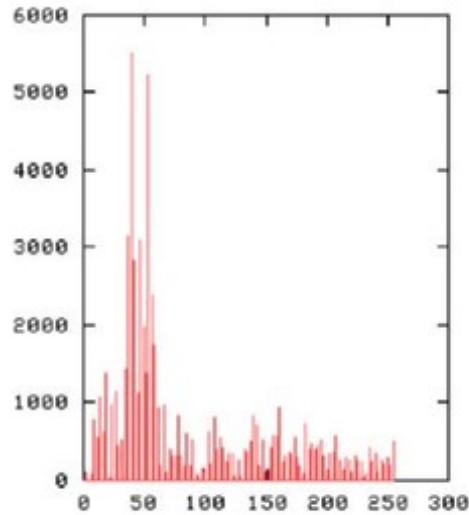


Image Representations via Histograms



Global histogram to represent
distribution of features

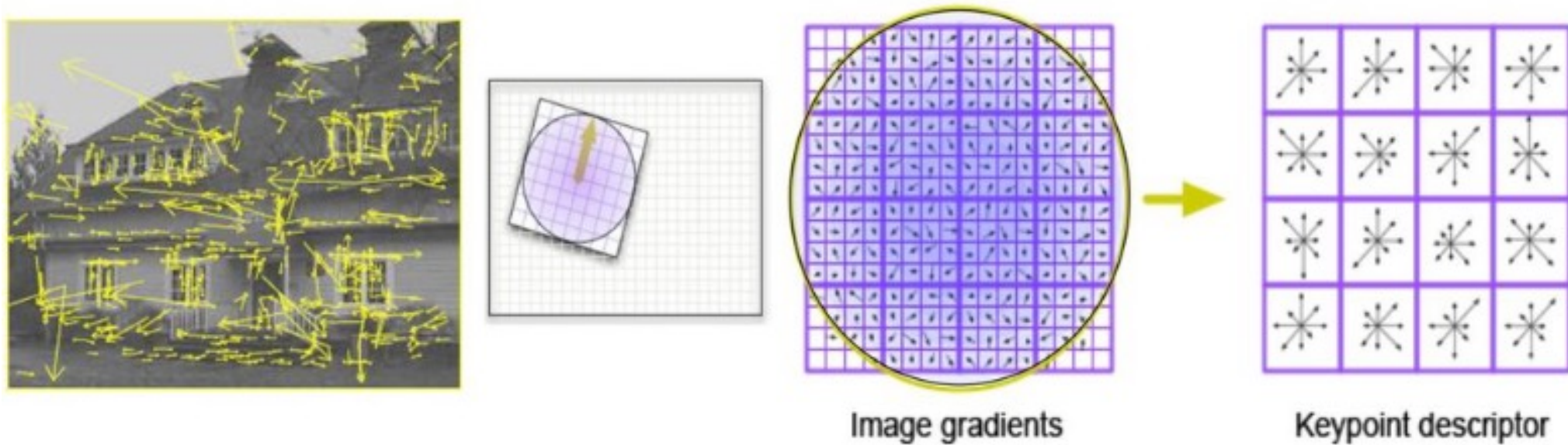
— Color, texture, depth, ...

Local histogram per detected point



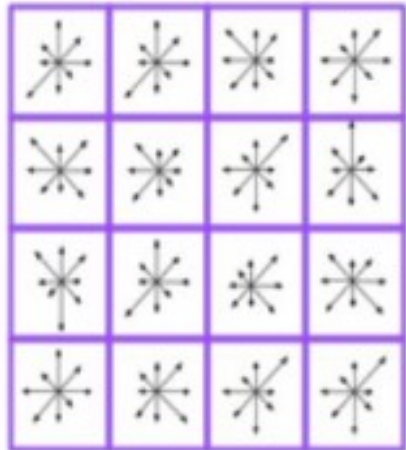
For What Things Do We Compute Histograms?

- Texture
- Local histogram of oriented gradient
- SIFT: Scale Invariant Feature Transform (~40k citations)

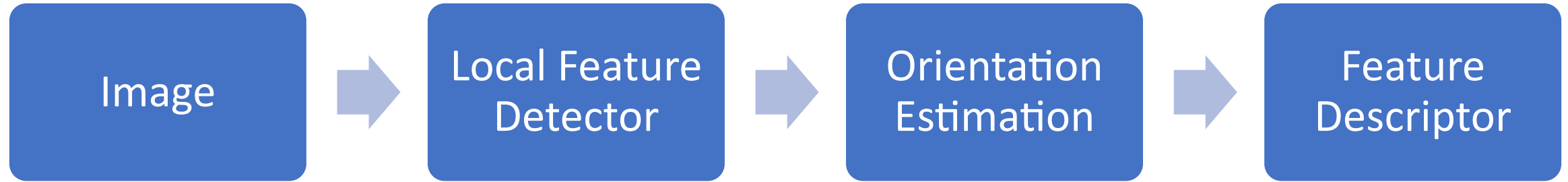


SIFT

- Keypoint:
 - Location
 - Scale
 - Orientation
- Descriptor
 - $4 \times 4 \times 8 = 128$ dimensional feature vector



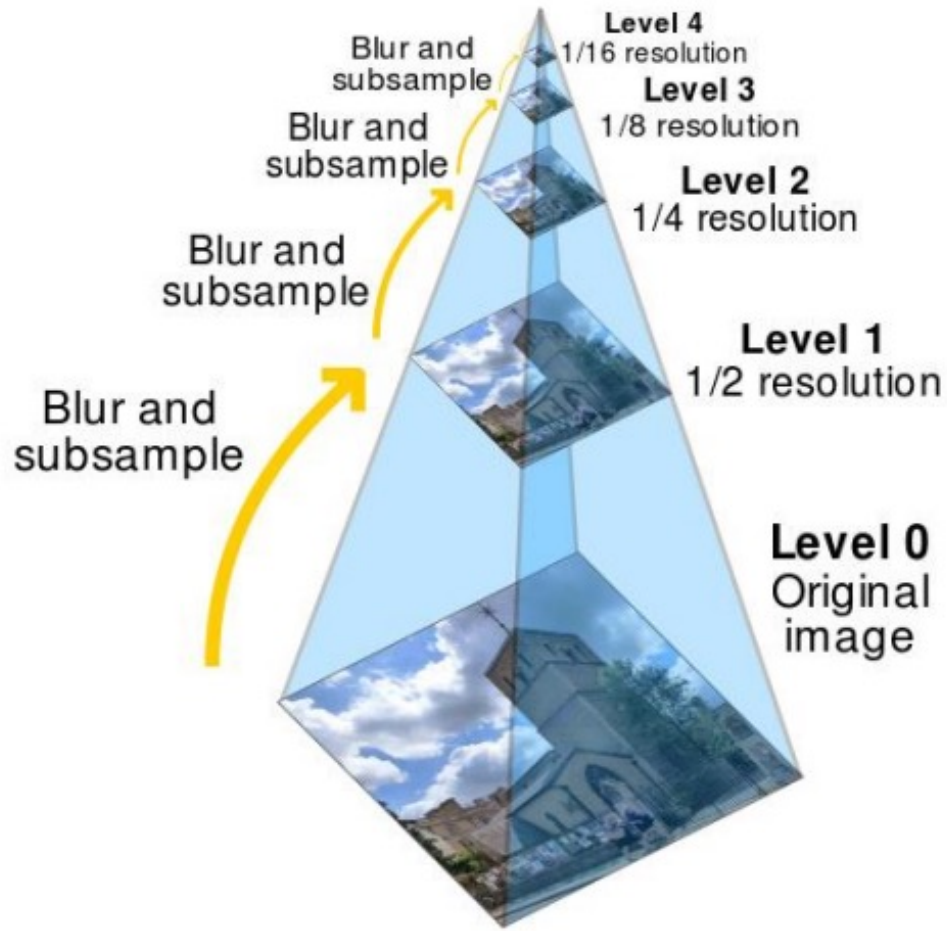
SIFT Pipeline



SIFT Pipeline Details

- Local feature detection
 - Find difference of Gaussian scale-space extrema
- Post-processing
 - Position interpolation
 - Discard low-contrast points
 - Eliminate points along edges
- Orientation estimation
- Descriptor extraction
 - Motivation: we want some sensitivity to spatial layout, but not too much, so blocks of histograms give us that

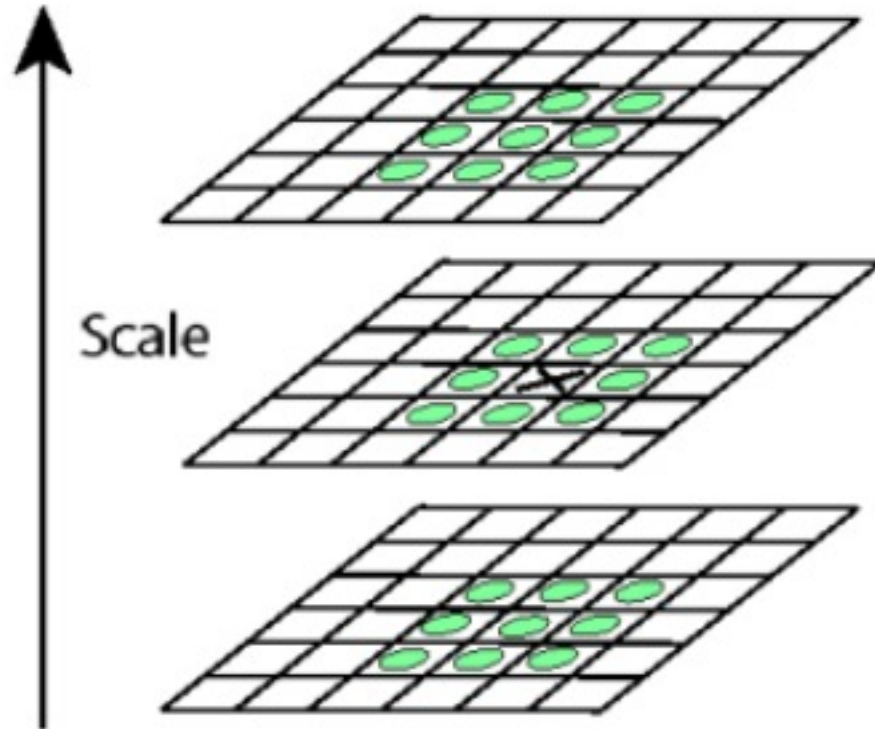
Scale Space Detection



GSS: Gaussian Scale Space is produced by iteratively convolving the last layer image with a Gaussian kernel.

DoGSS: DoG Scale Space is produced by subtracting neighboring GSS layers.

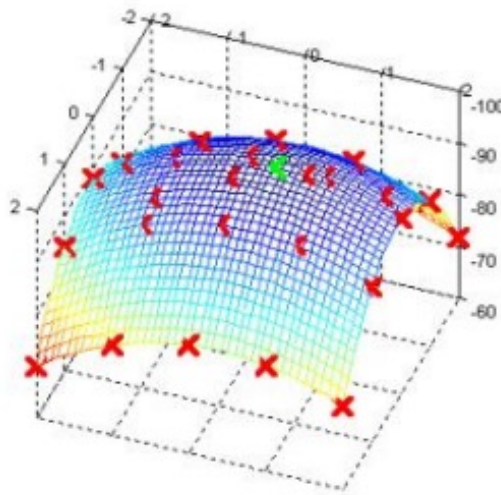
Non-Max Suppression in Scale Space



Search extrema in DoGSS to locate initial keypoints.

Keypoint Refinement

- Refine:
 - Fit a 3D (x, y, scale) curve to the initial keypoint, and find the peak in the curve as the refined keypoint



- Elimination:
 - Discard keypoints with low refined DoG response
 - Discard keypoints with high edge response

SIFT Orientation Normalization

- Compute orientation histogram:
 - Compute gradient magnitudes and orientations in a small window around the keypoint at the appropriate scale

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

$$m(x, y) = \sqrt{(L(x+1, y) - L(x-1, y))^2 + (L(x, y+1) - L(x, y-1))^2}$$

$$\theta(x, y) = \tan^{-1}((L(x, y+1) - L(x, y-1)) / (L(x+1, y) - L(x-1, y)))$$

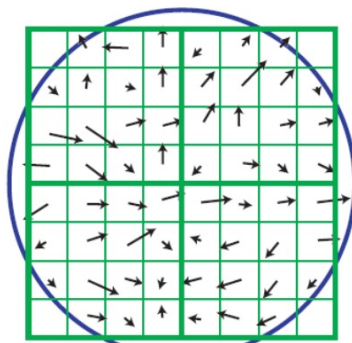
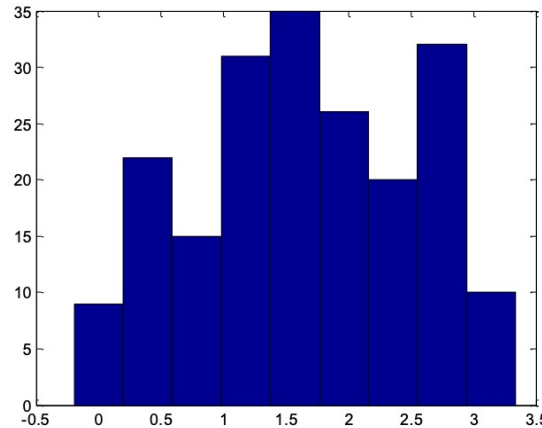
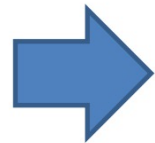


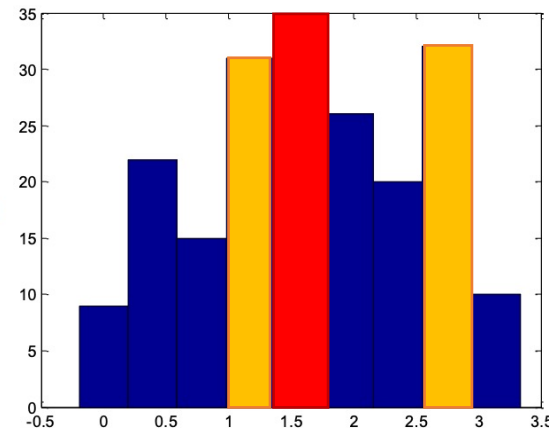
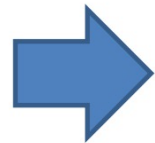
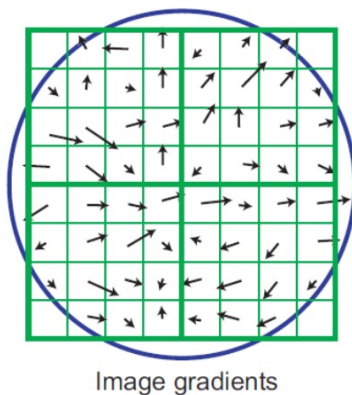
Image gradients



Histogram of gradient orientation – the bin-counts are weighted by gradient magnitudes and a Gaussian weighting function. Usually, 36 bins are chosen for the orientation.

SIFT Orientation Normalization

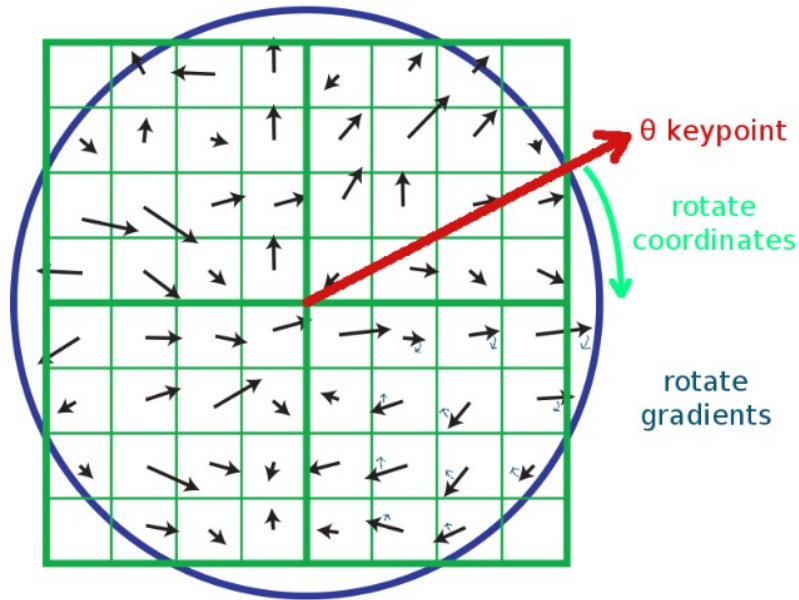
- Compute orientation histogram
- Select orientation
 - Assign **dominant orientation** as orientation of the keypoint
 - In case of multiple peaks or histogram entries more than 0.8 x peak, create a **separate descriptor** for each orientation (they will all have the same scale and location).



Histogram of gradient orientation – the bin-counts are weighted by gradient magnitudes and a Gaussian weighting function. Usually, 36 bins are chosen for the orientation.

SIFT Orientation Normalization

- Compute orientation histogram
- Select orientation
- Normalize:
 - Rotate to fixed orientation
 - Scale the window size based on the scale at which the point was found

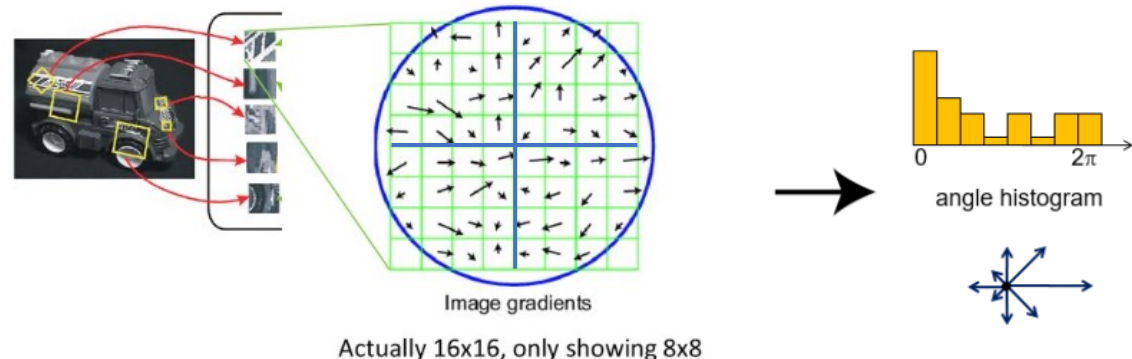


Keypoint Descriptors

- At this point, each keypoint has
 - Location
 - Scale
 - Orientation
- Next is to compute a descriptor for the local image region about each keypoint that is:
 - Highly distinctive
 - Invariant as possible to variations such as changes in viewpoint and illumination

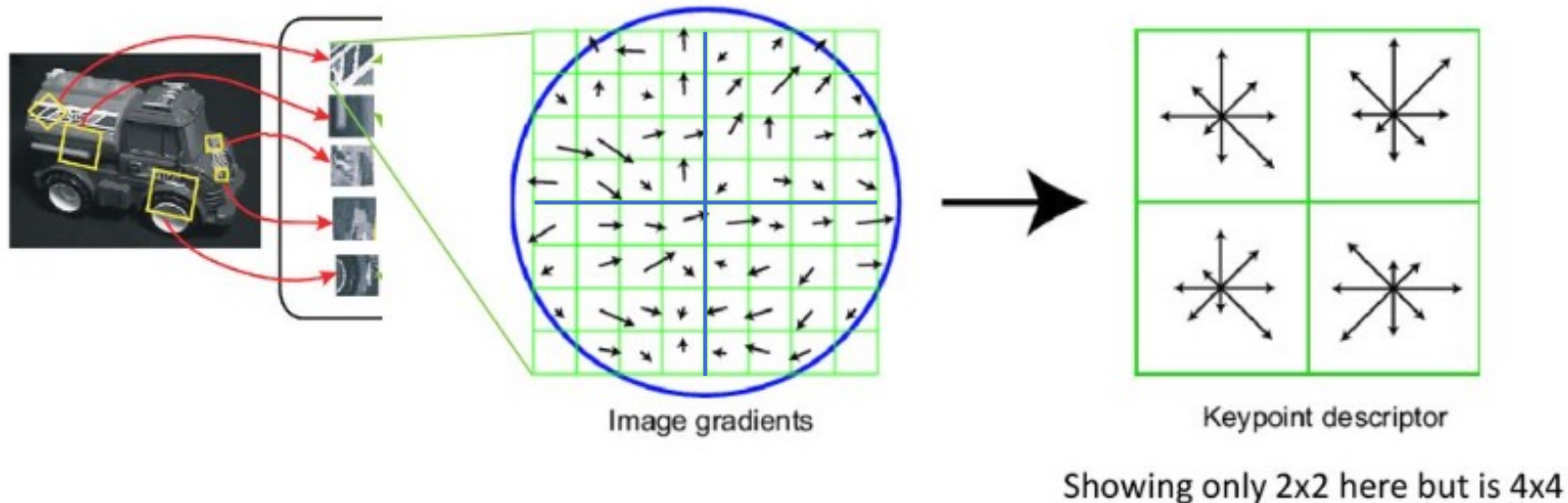
SIFT Descriptor Formation

- Compute on local 16x16 window around detection
 - Divide it into nxn cells (usually $n = 2$). Each cell is of size 4x4.
- Compute gradients weighed by a Gaussian of variance half the window (for smooth falloff)
 - Build a gradient orientation histogram in each cell. Each histogram entry is weighted by the gradient magnitude and a Gaussian weighting function with $\sigma = 0.5$ of window width.
- Sort each gradient orientation histogram bearing in mind the dominant orientation of the keypoint



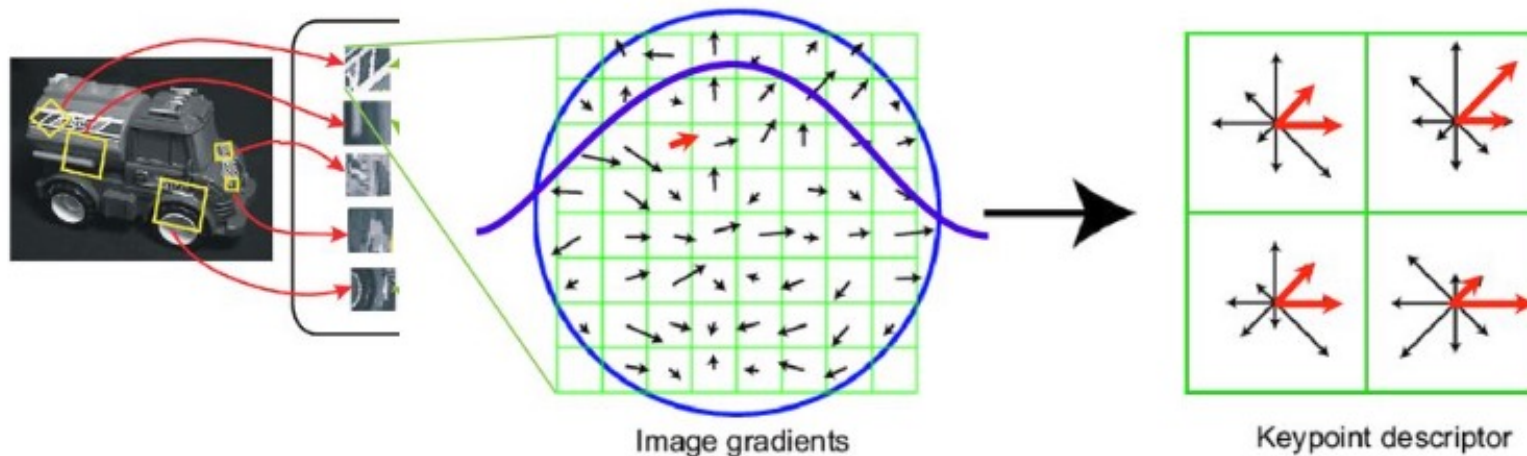
SIFT Vector Formation

- 4x4 array of gradient orientation histograms weighted by gradient magnitude
- Bin into 8 orientation * 4x4 array = 128 dimensions



Ensure Smoothness

- Gaussian weight the magnitudes
- Trilinear interpolation
 - A given gradient contributes to 8 bins: 4 in space * 2 in orientation



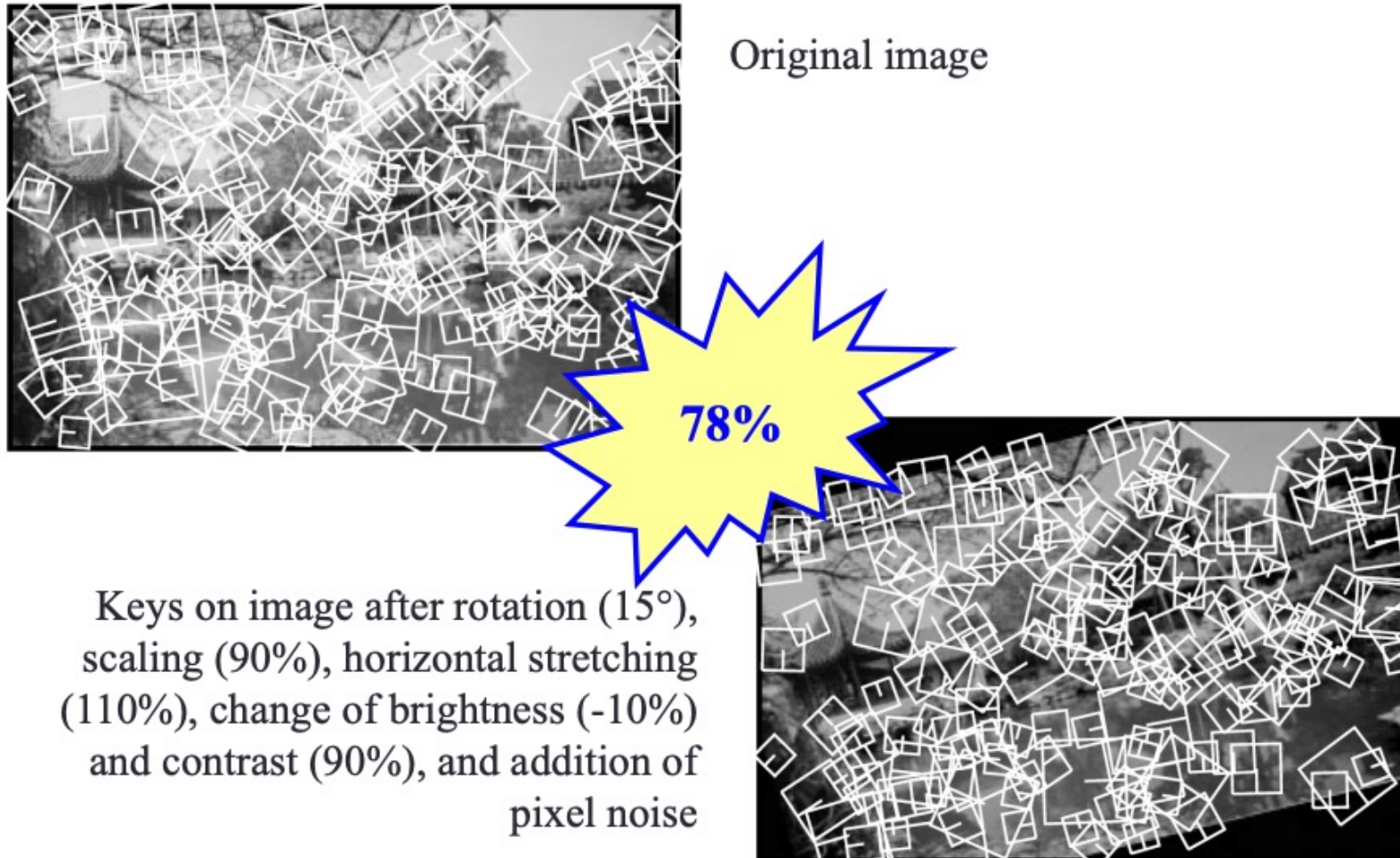
Reduce Effect of Illumination

- SIFT descriptor (so far) is not illumination invariant – the histogram entries are weighted by gradient magnitude.
 - Hence descriptor vector is normalized to unit magnitude. This will normalize scalar multiplicative intensity changes.
- 128-bin vector normalized to 1
- Threshold gradient magnitudes to avoid excessive influence of high gradients
 - After normalization, clamp gradient > 0.2
 - Renormalize

Evaluating SIFT descriptors

- Database images were subjected to rotation, scaling, affine stretch, brightness and contrast changes, and added noise. Feature point detectors and descriptors were compared before and after the distortions, and evaluated for:
 - Sensitivity to number of histogram orientations and subregions.
 - Stability to noise.
 - Stability to affine change.
 - Feature distinctiveness

Evaluating SIFT descriptors



SIFT in OpenCV

- `img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`
- `sift = cv2.xfeatures2d.SIFT_create()`
- `kp, des = sift.detectAndCompute(img, None)`
- `img=cv2.drawKeypoints(gray,kp,img)`
- `def extract_sift_features(X):`
 - `image_descriptors = []`
 - `sift = cv2.xfeatures2d.SIFT_create()`
 - `for i in range(len(X)):`
 - `_, des = sift.detectAndCompute(X[i], None)`
 - `image_descriptors.append(des)`
 - `return image_descriptors`



Other Local Features

- Handcrafted features:
 - SURF, BRIEF, BRISK, FREAK, FRIF, DAISY
- Learned features:
 - Traditional Learning: ASR, Brown's Descriptor
 - Deep Learning: LIFT, TFeat